Proposition

IQ-Learn Primer

Garg, Chakraborty, Cundy, Song, and Ermon (2021)

EconIRL Technical Notes

1 Problem and Motivation

IQ-Learn (Garg et al., 2021) addresses a gap between two approaches to imitation learning. Behavioral cloning (BC) is simple and stable but ignores the sequential structure of the problem: small errors compound as the agent drifts to states it never saw during training. Dynamics-aware methods such as GAIL (Ho & Ermon, 2016) correct this by framing IL as inverse RL, but require solving a min-max problem over reward and policy jointly. With sparse expert coverage, adversarial training is unstable: the discriminator overfits to the handful of observed state-action pairs and destabilizes policy updates.

IQ-Learn eliminates the min-max problem by learning a single Q-function that implicitly defines both reward and policy. The paper proves a one-to-one correspondence between the original IRL saddle point and a minimization over Q alone (Lemmas 3.1–3.2, Propositions 3.3–3.4). The resulting objective is a penalized conditional log-likelihood: the first term is BC's log-likelihood, and the chi-squared penalty on implied rewards adds Bellman structure. IQ-Learn is therefore a strict generalization of regularized BC. In the finite discrete DDC setting with known transitions, exact continuation values make this reduction algebraically exact. Unlike NFXP and GLADIUS, IQ-Learn does not enforce Bellman consistency as a hard constraint; the penalty encourages but does not guarantee it.

2 Model and Notation

A finite-state, finite-action MDP with state space $\mathcal{S}$, action set $\mathcal{A}(s)$, transitions $p(s'|s,a)$, reward $r(s,a)$, discount $\beta \in (0,1)$, and T1EV scale $\sigma > 0$. The key objects are as follows.

Table 1: MDP objects.

Symbol	Name	Definition
$Q(s,a)$	Choice-specific value	$r(s,a) + \beta \sum_{s'} p(s'\|s,a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s,a)/\sigma)$
$\pi(a\|s)$	Policy (CCP)	$\exp(Q(s,a)/\sigma) / \sum_{a'} \exp(Q(s,a')/\sigma)$
$\mathrm{td}(s,a)$	Temporal difference	$Q(s,a) - \beta \sum_{s'} p(s'\|s,a)\, V(s')$
α	Regularization	Controls penalty strength; $1/(4\alpha\sigma)$ is the penalty weight

The temporal difference $\mathrm{td}(s,a)$ is the reward implied by a given Q-function through the inverse Bellman operator. At the Bellman fixed point, $\mathrm{td}(s,a) = r(s,a)$. For a general Q that does not satisfy the Bellman equation, $\mathrm{td}(s,a)$ is the reward that would rationalize Q as a fixed point. The IQ-Learn objective is built from two algebraic identities that hold for any Q, not only for Q^*.

3 Core Theory: Penalized MLE Equivalence

Proposition 1 (Advantage equals scaled log CCP)**.** *For any $Q(s,a)$, define $V(s)$ and $\pi(a|s)$ from the table above. Then*

$$Q(s,a) - V(s) = \sigma \log \pi(a|s). \tag{1}$$

Proof. Take the log of the softmax: $\log \pi(a|s) = Q(s,a)/\sigma - \log(\sum_{a'} \exp(Q(s,a')/\sigma))$. Multiply by σ. □

Proposition 2 (Temporal difference equals implied reward)**.** *If Q satisfies the Bellman fixed point $Q = \Lambda_\sigma(Q)$, then $\mathrm{td}(s,a) = r(s,a)$.*

Proof. At the fixed point, $Q(s,a) = r(s,a) + \beta \sum_{s'} V(s')\, p(s'|s,a)$. Subtract $\beta \sum_{s'} V(s')\, p(s'|s,a)$ from both sides. □

The IQ-Learn chi-squared objective. Given expert observations $(s_i, a_i)_{i=1}^N$, IQ-Learn minimizes over Q:

$$\mathcal{L}(Q) = -\frac{1}{N}\sum_{i=1}^{N}\big[Q(s_i,a_i) - V(s_i)\big] + \frac{1}{4\alpha N}\sum_{i=1}^{N}\big[\mathrm{td}(s_i,a_i)\big]^2. \tag{2}$$

Step 1. By Proposition 1, the first term equals $-(\sigma/N)\sum_i \log \pi(a_i|s_i) = -(\sigma/N)\,\mathrm{LL}(Q)$, the scaled negative conditional log-likelihood.

Step 2. Writing $\hat{r}_i \equiv \mathrm{td}(s_i, a_i)$, the second term is $(4\alpha N)^{-1}\sum_i \hat{r}_i^2$, the average squared implied reward.

Step 3. Combining and multiplying by $-N/\sigma$ converts minimization of $\mathcal{L}$ into maximization of:

$$\boxed{\mathrm{LL}(Q) \;-\; \frac{1}{4\alpha\sigma}\sum_{i=1}^{N}\hat{r}_i^{\,2}} \tag{3}$$

where $\hat{r}_i = Q(s_i, a_i) - \beta\sum_{s'} V(s')\,p(s'|s_i, a_i)$ and $V(s) = \sigma \log \sum_a \exp(Q(s,a)/\sigma)$.

Expression (3) is a penalized conditional log-likelihood. The first term is the multinomial logit log-likelihood with Q-values as utilities. The second term is an ℓ^2 penalty on the Bellman-implied reward, weighted by the expert occupancy. The penalty is quadratic in $\hat{r}$ but not in Q (because V depends on Q through the logsumexp). No inner Bellman solve appears because V is an explicit function of Q.

Identification. Multiple reward functions rationalize the same policy. For any potential $h : \mathcal{S} \to \mathbb{R}$, the shaped reward $r'(s,a) = r(s,a) + h(s) - \beta\sum_{s'} h(s')\,p(s'|s,a)$ yields the same optimal policy (Ng et al., 1999). IQ-Learn does not resolve this non-identification. The chi-squared regularizer selects the element with smallest implied reward at expert-visited states. This has no economic interpretation, unlike the anchor normalizations of AAIRL (Lee et al., 2026) which set the exit payoff to zero. As $\alpha \to \infty$ the penalty vanishes and the objective reduces to standard conditional log-likelihood maximization, recovering BC as a special case with no Bellman structure.

Comparison to GLADIUS. Both IQ-Learn and GLADIUS combine a log-likelihood term with a Bellman-consistency penalty. The key difference is how the continuation value $\mathbb{E}[V(s')|s,a]$ is handled. IQ-Learn with known transitions computes it exactly. GLADIUS works model-free and introduces a dual variable $\zeta(s,a) \approx \mathbb{E}[V(s')|s,a]$ to correct the double-sampling bias. GLADIUS also explicitly enforces Bellman consistency and has global convergence guarantees via the PL condition, while IQ-Learn does not enforce or guarantee Bellman satisfaction.

4 Estimation Algorithm

Algorithm 1: IQ-Learn (tabular DDC setting)

Input: Expert data $\{(s_i, a_i)\}_{i=1}^N$, transitions $p(s'|s,a)$, discount β, scale σ, regularization α

Initialize $Q(s,a)$ for all (s,a) (e.g., zeros or from behavioral cloning)

repeat

`// Forward pass: compute` V `and` π `from current` Q

$V(s) \leftarrow \sigma \log \sum_a \exp(Q(s,a)/\sigma)$ for all s

$\pi(a|s) \leftarrow \exp(Q(s,a)/\sigma) / \sum_{a'} \exp(Q(s,a')/\sigma)$ for all (s,a)

`// Implied reward at expert observations`

$\hat{r}_i \leftarrow Q(s_i, a_i) - \beta \sum_{s'} p(s'|s_i, a_i)\, V(s')$ for $i = 1, \ldots, N$

`// Penalized log-likelihood`

$\mathrm{LL} \leftarrow \sum_{i=1}^N \log \pi(a_i|s_i)$

$\mathcal{J}(Q) \leftarrow \mathrm{LL} - \frac{1}{4\alpha\sigma} \sum_{i=1}^N \hat{r}_i^2$ `// Eq. (3)`

`// Update` Q `by gradient ascent`

$Q \leftarrow Q + \eta \nabla_Q \mathcal{J}(Q)$ `// or L-BFGS-B`

until *convergence*

`// Extract structural objects`

$\hat{r}(s,a) \leftarrow Q(s,a) - \beta \sum_{s'} p(s'|s,a)\, V(s')$ for all (s,a)

$\hat{\pi}(a|s) \leftarrow \mathrm{softmax}(Q(s,\cdot)/\sigma)$

Output: $\hat{Q}$, $\hat{r}$, $\hat{\pi}$

Implementation notes. In the tabular setting with known transitions, the gradient $\nabla_Q \mathcal{J}$ is computed by automatic differentiation through the logsumexp. No inner Bellman loop is needed. The EconIRL implementation uses L-BFGS-B for the outer optimization. In continuous spaces where p is unknown, the original IQ-Learn paper replaces the exact continuation value with sampled next states and adds a separate policy network, but these are implementation details for the continuous case. The regularization parameter α controls how tightly the Bellman equation is enforced: $\alpha \to 0$ forces $\hat{r} \to 0$ at expert states, $\alpha \to \infty$ removes the penalty and reduces to unconstrained logit with no structural content.

5 Results

Table 2: Low-data stability benchmark: 5 trajectories (500 obs), 3 seeds. Policy acc. vs. oracle NFXP (500 trajectories, $\beta = 0.95$). OOD: states 70–89 (high-mileage bins rarely visited in training). Cos-sim: recovered vs. true $(\theta_1, \theta_2) = (0.001, 3.0)$; N/A for non-parametric. Mean ± std.

Method	Policy acc. (%)	OOD acc. (%)	Reward cos-sim	Time (s)
IQ-Learn	98.9 ± 0.0	100.0 ± 0.0	1.000 ± 0.000	1
MCE-IRL	100.0 ± 0.0	100.0 ± 0.0	1.000 ± 0.000	8
GAIL	1.1 ± 0.0	0.0 ± 0.0	—	47
BC	98.9 ± 0.0	100.0 ± 0.0	—	<1

Four methods run on five expert trajectories of the 90-state Rust bus, three seeds, policy accuracy against oracle NFXP (500 trajectories).

GAIL collapses: with only five trajectories the discriminator covers ≈60 of 90 state-action pairs and produces a destabilizing reward signal, reaching 1.1% at 56 seconds. IQ-Learn achieves 98.9% with perfect reward recovery in 1.1 seconds. IQ-Learn and BC both reach 98.9%—this is the expected result since IQ-Learn is a generalization of regularized BC; on a task simple enough for BC, the two match on policy accuracy. The structural difference is reward recovery: IQ-Learn returns $(\hat{\theta}_1, \hat{\theta}_2) = (0.001, 3.0)$ with cosine similarity 1.000; BC recovers no reward. MCE-IRL achieves 100% via full soft value iteration but takes 9.6 seconds. On OOD states 70–89, all three stable methods reach 100%; GAIL collapses there too.

Counterfactuals. The recovered policy $\hat{\pi}(a|s) = \text{softmax}(Q/\sigma)$ is identified: it is invariant to additive potential shaping $r' = r + h(s) - \beta \sum_{s'} h(s')p(s'|s,a)$. Action rankings at a given state and behavioral prediction under the *current* environment are valid.

Four uses are *not* valid. (1) Policy under new transitions P': requires solving the Bellman equation under P' with an identified reward; the IQ-Learn Q is not a Bellman fixed point. (2) Welfare comparisons: reward scale is set by the chi-squared regularizer (minimum $\|\hat{r}\|^2$), not by an economic anchor. (3) Cross-state reward comparisons: the additive potential $h(s)$ shifts reward independently per state, so $\hat{r}(s,a)$ is not identified across states. (4) Impulse-response under parameter changes: requires structural Bellman consistency. For all four, use NFXP or MCE-IRL.

6 EconIRL Implementation

The estimator is `IQLearnEstimator` in `src/econirl/estimation/iq_learn.py`. Run `iq_learn_run.py` in this folder to regenerate results.

Table 3: Code mapping.

Step	Method	Operation
Forward pass	`_compute_values()`	$V = \sigma \,\text{logsumexp}(Q/\sigma)$
Implied reward	`_compute_td()`	$\hat{r} = Q - \beta\, P\, V$
Penalized LL	`_objective()`	$\text{LL} - (4\alpha\sigma)^{-1} \sum \hat{r}^2$
Optimize	`minimize_lbfgsb()`	L-BFGS-B over Q-table

```
from econirl.environments.rust_bus import RustBusEnvironment
from econirl.estimation.iq_learn import IQLearnEstimator, IQLearnConfig
from econirl.preferences.linear import LinearUtility

env = RustBusEnvironment(operating_cost=0.001, replacement_cost=3.0,
                         num_mileage_bins=90, discount_factor=0.95)
panel = env.generate_panel(n_individuals=5, n_periods=100, seed=42)

iq = IQLearnEstimator(config=IQLearnConfig(alpha=1.0))
result = iq.estimate(panel=panel, utility=LinearUtility.from_environment(env),
                     problem=env.problem_spec, transitions=env.transition_matrices)
```

Table 4: IQ-Learn in context.

	NFXP	CCP	IQ-Learn	GLADIUS	AAIRL
Bellman enforced	✓	✓		✓	
Transition-free				✓	✓
Inner loop	✓				
Global convergence	✓	✓		✓	
Identification	parametric	parametric	regularizer	anchor	anchor

Comparison across estimators. **References.**

Garg, D., Chakraborty, S., Cundy, C., Song, J., and Ermon, S. (2021). IQ-Learn: Inverse soft-Q learning for imitation. *NeurIPS 35*.

Kang, E.H., Yoganarasimhan, H., and Jain, L. (2025). An empirical risk minimization approach for offline inverse RL and DDC. arXiv:2502.14131.

Lee, P.S., Sudhir, K., and Wang, T. (2026). Modeling serialized content consumption: Adversarial IRL for DDC. Yale SOM.

Ng, A., Harada, D., and Russell, S. (1999). Policy invariance under reward transformations. *ICML*.

Rust, J. (1987). Optimal replacement of GMC bus engines. *Econometrica* 55(5):999–1033.